



OpenMP - Cláusulas

- comportamento variáveis
- escalonamento

Professora: Carla Santana
carla.santana@imd.ufrn.br

Última aula

- OpenMP – Região Crítica
- Diretivas
 - critical
 - atomic
 - reduction





Cláusula shared, default, private

- **shared**: a variável é acessada por todas as threads
- **private**: cada thread possui sua própria cópia da variável, não visível para as demais
- **default (shared | none)**
 - **shared**: as variáveis não especificadas explicitamente serão compartilhadas por padrão.
 - **none**: todas as variáveis devem ter seu comportamento definido explicitamente (como private ou shared), ou serem declaradas dentro do escopo da diretiva.



Cláusula shared, default

```
#pragma omp parallel for shared (a ,n) default ( none )  
for (i = 0; i < n; i ++ ) {  
    a[i] += i;  
}
```

O `i` por padrão é privado.
Variável de índice do laço mais
externo é sempre privada por
padrão



Cláusula private, firstprivate, lastprivate

- **private**: cada thread possui sua própria cópia da variável, não visível para as demais
- **firstprivate**: cria cópias privadas das variáveis para cada thread, inicializando-as com o valor que tinham antes da região paralela (geralmente da thread mestre).
- **lastprivate**: a variável privada assume, ao final da execução, o valor correspondente à última iteração do loop (de acordo com a ordem sequencial).



Cláusula private, firstprivate, lastprivate

```
void main (int argc, char *argv[]) {  
    int i, qtd_threads = 4,  
        n = 2*qtd_threads,  
        a = 1000;  
  
    omp_set_num_threads(qtd_threads);  
  
    #pragma omp parallel  
    {  
        printf("Thread %d: \  
            valor inicial de a = %d\n", \  
            omp_get_thread_num(), a);  
  
        #pragma omp for  
        for (int i = 0; i < n; i++) {  
            a++;  
            printf("Thread %d: \  
                valor de a = %d \  
                na iteracao =%d\n", \  
                omp_get_thread_num(), a, i);  
        }  
        printf("Thread %d: \  
            valor final de a = %d\n", \  
            omp_get_thread_num(), a);  
    }  
    printf("Thread master: a = %d\n", a);  
}
```



Cláusula private, firstprivate, lastprivate

```
void main (int argc, char *argv[]) {  
    int i, qtd_threads = 4,  
        n = 2*qtd_threads,  
        a = 1000;  
  
    omp_set_num_threads(qtd_threads);  
  
    #pragma omp parallel private(a)  
    {  
        printf("Thread %d: \  
            valor inicial de a = %d\  
            omp_get_thread_num(), a);  
  
        #pragma omp for  
        for (int i = 0; i < n; i++) {  
            a++;  
            printf("Thread %d: \  
                valor de a = %d \  
                na iteracao =%d\  
                omp_get_thread_num(), a, i);  
        }  
        printf("Thread %d: \  
            valor final de a = %d\  
            omp_get_thread_num(), a);  
    }  
    printf("Thread master: a = %d\  
}
```



Cláusula private, firstprivate, lastprivate

```
void main (int argc, char *argv[]) {  
    int i, qtd_threads = 4,  
        n = 2*qtd_threads,  
        a = 1000;  
  
    omp_set_num_threads(qtd_threads);  
  
    #pragma omp parallel firstprivate(a)  
    {  
        printf("Thread %d: \  
            valor inicial de a = %d\  
            \", \  
            omp_get_thread_num(), a);  
  
        #pragma omp for  
        for (int i = 0; i < n; i++) {  
            a++;  
            printf("Thread %d: \  
                valor de a = %d \  
                na iteracao =%d\  
                \", \  
                omp_get_thread_num(), a, i);  
        }  
        printf("Thread %d: \  
            valor final de a = %d\  
            \", \  
            omp_get_thread_num(), a);  
    }  
    printf("Thread master: a = %d\  
    \", a);  
}
```



Cláusula private, firstprivate, lastprivate

```
void main (int argc, char *argv[]) {  
    int i, qtd_threads = 4,  
        n = 2*qtd_threads,  
        a = 1000;  
  
    omp_set_num_threads(qtd_threads);  
  
    #pragma omp parallel  
    {  
        printf("Thread %d: \  
            valor inicial de a = %d\  
            \", \  
            omp_get_thread_num(), a);  
  
        #pragma omp for lastprivate(a)  
        for (int i = 0; i < n; i++) {  
            a++;  
            printf("Thread %d: \  
                valor de a = %d \  
                na iteracao = %d\  
                \", \  
                omp_get_thread_num(), a, i);  
        }  
        printf("Thread %d: \  
            valor final de a = %d\  
            \", \  
            omp_get_thread_num(), a);  
    }  
    printf("Thread master: a = %d\  
    \", a);  
}
```



Cláusula private, firstprivate, lastprivate

```
void main (int argc, char *argv[]) {  
    int i, qtd_threads = 4,  
        n = 2*qtd_threads,  
        a = 1000;  
  
    omp_set_num_threads(qtd_threads);  
  
    #pragma omp parallel  
    {  
        printf("Thread %d: \  
            valor inicial de a = %d\  
            \", \  
            omp_get_thread_num(), a);  
  
        #pragma omp for firstprivate (a)  
        lastprivate(a)  
        for (int i = 0; i < n; i++) {  
            a++;  
            printf("Thread %d: \  
                valor de a = %d \  
                na iteracao =%d\  
                \", \  
                omp_get_thread_num(), a, i);  
        }  
        printf("Thread %d: \  
            valor final de a = %d\  
            \", \  
            omp_get_thread_num(), a);  
    }  
    printf("Thread master: a = %d\  
    \", a);  
}
```



Cláusula private, firstprivate, lastprivate

```
void main (int argc, char *argv[]) {  
    int i, qtd_threads = 4,  
        n = 2*qtd_threads,  
        a = 1000;  
  
    omp_set_num_threads(qtd_threads);  
  
    #pragma omp parallel  
    {  
        printf("Thread %d: \  
            valor inicial de a = %d\  
            \n", \  
                omp_get_thread_num(), a);  
  
        #pragma omp for reduction (+:a)  
        for (int i = 0; i < n; i++) {  
            a++;  
            printf("Thread %d: \  
                valor de a = %d \  
                na iteracao =%d\  
                \n", \  
                    omp_get_thread_num(), a, i);  
        }  
        printf("Thread %d: \  
            valor final de a = %d\  
            \n", \  
                omp_get_thread_num(), a);  
    }  
    printf("Thread master: a = %d\  
    \n", a);  
}
```

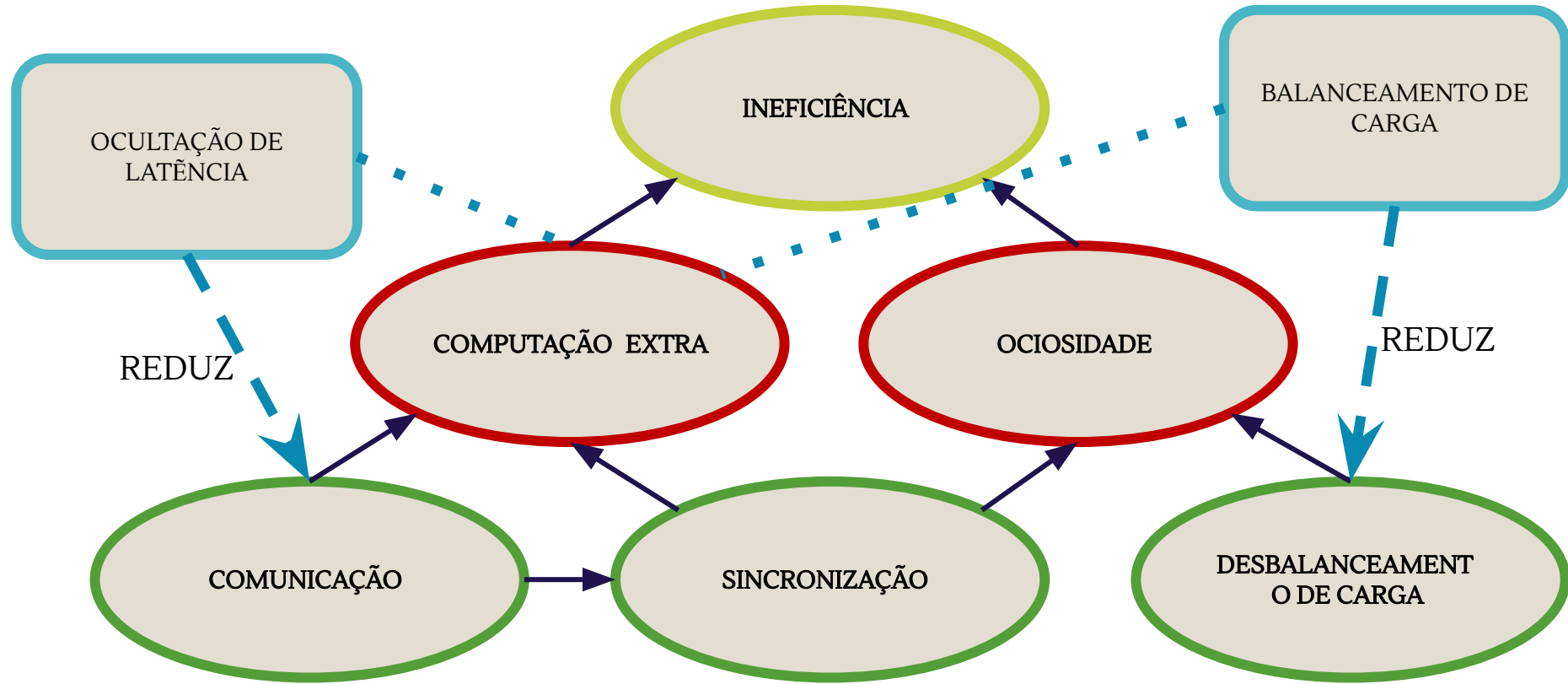


Medindo o tempo

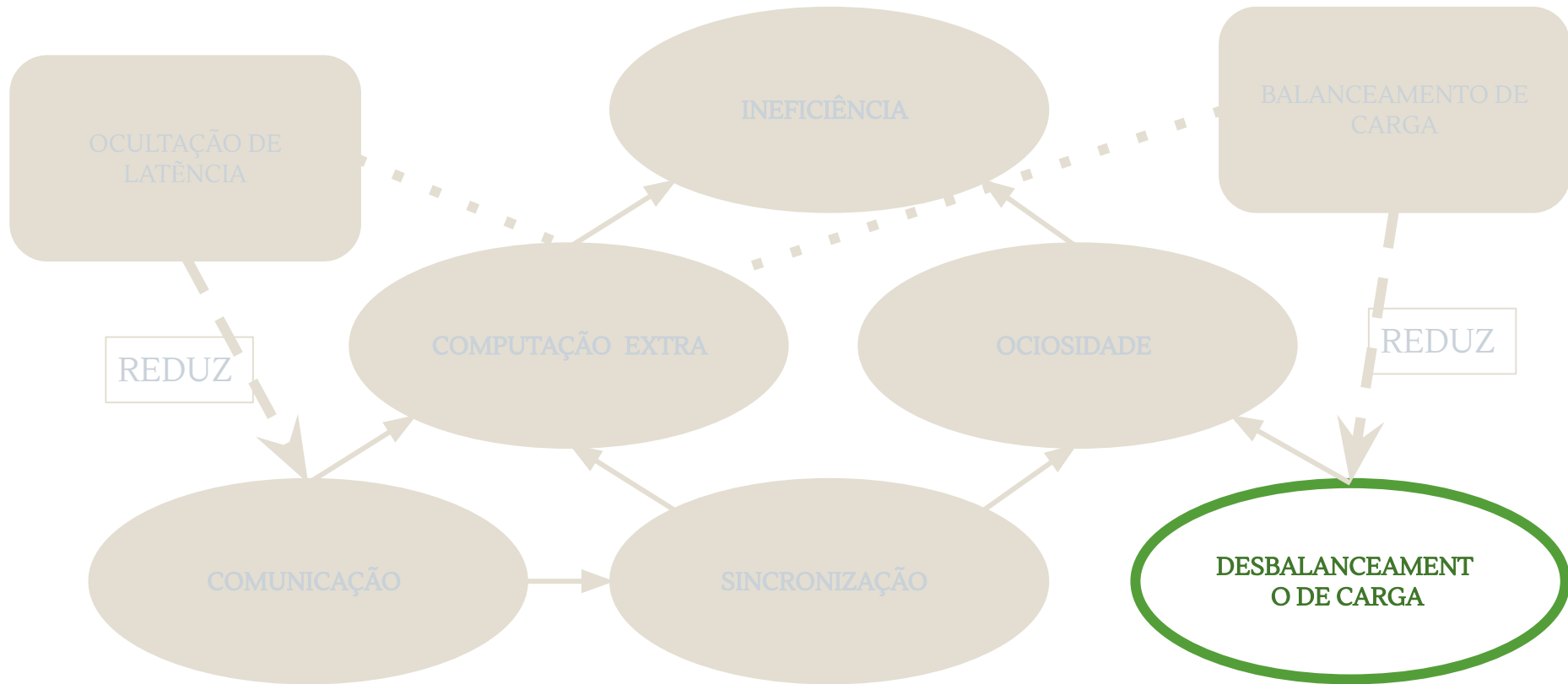
```
#include <stdio.h>
#include <omp.h>
int main ( int argc , char * argv []) { /* omp_parallel .c */
    double inicio , fim ;
    inicio = omp_get_wtime () ;
    #pragma omp parallel num_threads (4)
    {
        int tid = omp_get_thread_num () ;
        #pragma omp for
        for (int i = 0; i < 12; i++)
        {
            printf (" Alo da thread %d. fazendo o trabalho %d \n", tid, i );
        }
    }
    fim = omp_get_wtime () ;
    printf ("Tempo gasto na execução = %f segundos \n", fim - inicio);
    return 0;
}

// Compilar
// gcc -o my_omp_parallel_program omp_parallel.c -fopenmp
// Executar
// ./my_omp_parallel_program
```

Por que é mais difícil programar em paralelo?



Por que é mais difícil programar em paralelo?





Cláusula schedule

- A cláusula schedule permite uma variedade de opções para especificar quais threads executarão as iterações dos laços.

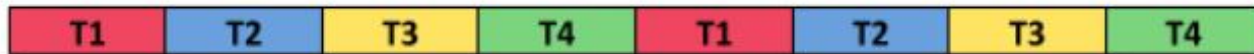
schedule (tipo[, chunk_size])

- Tipo:
 - static
 - dynamic
 - guided
 - auto
 - runtime
- **chunk_size** é uma expressão inteira com valor positivo.



Cláusula `schedule - static`

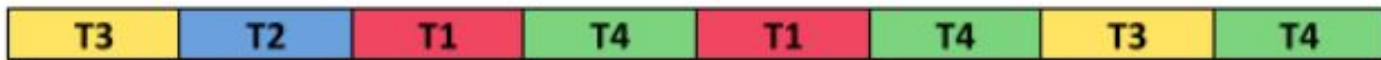
- No escalonamento **static**,
- Sem a especificação de **chunk_size**:
 - O espaço de iteração é dividido em pedaços (aproximadamente) iguais e cada pedaço é atribuído a cada thread (escalonamento em bloco).
- Se o valor de **chunk_size** for especificado:
 - O espaço de iteração é dividido em pedaços, cada um com **chunk_size** iterações, e os pedaços são atribuídos ciclicamente a cada thread (escalonamento cíclico por bloco).





Cláusula schedule - dynamic

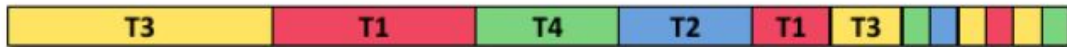
- O escalonamento dynamic divide o espaço de iteração em pedaços com `chunk_size` iterações, e os atribui para as threads com uma política “**primeiro a chegar, primeiro a ser atendido**”, isto é, se uma thread terminou um pedaço, ela receberá o próximo pedaço da lista.
- Quando nenhum valor de `chunk_size` for especificado, o valor padrão assumido é 1





Cláusula schedule - guided

- O escalonamento guided é similar ao dynamic, **mas os pedaços iniciam grandes e se tornam menores exponencialmente.**
- O tamanho do próximo pedaço é (com arredondamento para cima) **o número de iterações restantes dividido pelo número de threads.**
- O valor `chunk_size` especifica o tamanho mínimo dos pedaços
 - **exceto para o pedaço com as últimas iterações, que pode ser menor.**
- Quando nenhum valor de `chunk_size` for especificado, o padrão é igual a 1



```
#pragma omp parallel for schedule(guided, 4)
for (int i = 0; i < 100; i++) {
    trabalho(i);
}
// início:   blocos grandes → 25, 18, 14...
// meio:     blocos médios → 10, 8...
// final:    blocos pequenos → 4, 4, 4...
```



Cláusula schedule - auto

- O escalonamento auto dá liberdade para o ambiente de execução escolher qual método de escalonamento será utilizado.
- Isso pode ser útil, por exemplo, quando **o laço paralelo é executado diversas vezes**, permitindo ao ambiente de execução convergir para um tipo de escalonamento com um bom balanceamento de carga e com baixa sobrecarga.



Cláusula schedule - runtime

- O escalonamento runtime **delega a escolha do escalonamento para o momento da execução do programa**
- Podendo ser determinado pelo valor da variável de ambiente `OMP_SCHEDULE` ou pela chamada à função `omp_set_schedule()`.
- Quando presente, a variável de ambiente `OMP_SCHEDULE` especifica o tipo de escalonamento a ser utilizado nos laços.
- Esses tipos podem ser somente `static`, `dynamic`, `guided` ou `auto`.

```
export OMP_SCHEDULE = tipo [, tam ]  
export OMP_SCHEDULE = "guided ,4"  
export OMP_SCHEDULE = "static ,1000"
```



Cláusula schedule

- Qual tipo de escalonamento deve ser utilizado?
 - **static**: melhor para laços balanceados – menor sobrecarga;
 - **static, n**: melhor para laços com desbalanceamento suave;
 - **dynamic**: útil se as iterações têm grande variação de carga, mas acaba com a localidade espacial dos dados e tem o custo adicional de troca de threads;
 - **guided**: frequentemente menos cara que dynamic, mas tenha cuidado com laços onde as primeiras iterações são as mais demoradas;
 - **auto**: pode ser útil se o laço é executado diversas vezes ao longo da execução do programa;
 - Use **runtime** para uma experimentação adequada.

Tarefa 7



Implemente diferentes versões de um programa para o cálculo de quantos números primos existem entre 2 e um valor máximo n . O programa deve ser paralelizado usando OpenMP.

Devem ser implementadas as seguintes versões:

- A variável do contador é apenas **privado**
- A variável do contador é apenas **firstprivado**
- A variável do contador é apenas **lastprivado**
- A variável do contador é **firstprivate e lastprivado**
- Uso **default(none)**
- Paralela utilizando **reduction**

Após a implementação, compare o comportamento das diferentes versões e discuta os resultados, identificando e justificando qual delas apresenta o melhor desempenho. A partir da versão selecionada como a melhor, implemente as seguintes versões considerando diferentes estratégias de escalonamento.

- Escalonador **static**
- Escalonador **dynamic**
- Escalonador **guided**
- Escalonador **auto**
- Escalonador **runtime**

Analise os tempos médios de execução de cada versão à medida que o tamanho de n aumenta, utilizando gráficos para apresentar e analisar os resultados. **Por fim, discuta o comportamento observado para cada escalonador, destacando suas diferenças de desempenho e possíveis razões para os resultados obtidos.**